ETS

PEMROGRAMAN ROBOTIKA DENGAN KIT IMCLAB MENGGUNAKAN PYTHON



Dosen Pengampu:

Dr. Basuki Rahmat, S.Si. MT

Disusun oleh:

Arif Nur Cahyo (23081010192)

ROBOTIKA A – 081

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL " VETERAN"

JAWA TIMUR

2026

DAFTAR ISI

DAFTAR ISI i
DAFTAR GAMBAR ii
BAB I PERINTAH DAN KODE PROGRAM 1
1.1 Daftar Perintah Program Python 1
1.2 Kode Program 2
BAB II HASIL DAN PEMBAHASAN 5
2.1 Deteksi Port Otomatis dan Inisialisasi Program 5
2.2 Pengujian Perintah ‘1’ dan ‘0’ 5
2.3 Kondisi Perangkat Sebelum Roda Berputar (0) 6
2.4 Kondisi Perangkat Saat Roda Berputar (1) 7
2.5 Pengujian Perintah ‘+’ (Menaikkan Kecepatan) 8
2.6 Pengujian Perintah ‘-’ (Menurunkan Kecepatan) 9
2.7 Pengujian Perintah ‘PWM:220’ (Sintaks Keliru) 10
2.8 Pengujian Perintah Kosong (Validasi Masukan) 11
2.9 Rangkuman Hasil Pengujian 11
BAB III PENUTUP 13
3.1 Kesimpulan 13
3.2 Saran 13

DAFTAR GAMBAR

Gambar 1. Sel program dan proses deteksi port otomatis pada Jupyter Notebook..5
Gambar 2. Hasil perintah ‘1’ dan ‘0’ pada Jupyter Notebook 6
Gambar 3. Kondisi perangkat sebelum roda berputar (motor OFF) 7
Gambar 4. Kondisi perangkat saat roda berputar (motor ON).....8
Gambar 5. Hasil perintah ‘+’ sebanyak dua kali.....9
Gambar 6. Hasil perintah ‘-’ setelah kecepatan dinaikkan 10
Gambar 7. Penolakan perintah ‘PWM:220’ oleh program Python.....10
Gambar 8. Penolakan masukan kosong oleh validasi program 11

BAB I PERINTAH DAN KODE PROGRAM

Pada praktikum ini pengendalian motor DC pada KIT IMCLAB tidak lagi dilakukan melalui Serial Monitor Arduino IDE, melainkan melalui program Python yang dijalankan pada Jupyter Notebook dengan berkas Tes_IMCLab_naik_turun.ipynb. Program Python berperan sebagai sisi host yang mengirimkan perintah ke board ESP32, sedangkan firmware pada ESP32 tetap menggunakan program imclab_naik_turun.ino yang bertugas mengatur duty cycle PWM dan menggerakkan motor.

Komunikasi dilakukan menggunakan pustaka pyserial pada kecepatan 115200 baud. Keunggulan utama program ini adalah kemampuan mendeteksi port ESP32 secara otomatis, sehingga pengguna tidak perlu menuliskan nama port secara manual. Deteksi dilakukan dengan memeriksa pasangan VID:PID serta kata kunci deskripsi perangkat seperti CP210x, CH340, dan Silicon Labs, kemudian memberikan skor pada setiap port yang terdeteksi.

1.1 Daftar Perintah Program Python

Program menyediakan enam perintah yang dapat dimasukkan pada kotak input interaktif Jupyter Notebook sebagai berikut.

No	Perintah	Fungsi
1	‘1’	Memanggil motor_on(), mengirim karakter ‘1’ ke ESP32
2	‘0’	Memanggil motor_off(), mengirim karakter ‘0’ ke ESP32
3	‘+’	Menaikkan kecepatan sebesar 10 satuan duty cycle
4	‘-’	Menurunkan kecepatan sebesar 10 satuan duty cycle
5	‘p<val>’	Set PWM langsung, contoh p150; dikirim sebagai ‘PWM:150’
6	‘q’	Keluar dari mode interaktif dan menutup koneksi serial

Perlu diperhatikan bahwa sintaks pada sisi Python berbeda dengan sintaks pada

firmware. Pengguna mengetikkan p150, kemudian fungsi set_pwm() yang mengubahnya menjadi perintah PWM:150 sebelum dikirimkan ke ESP32.

1.2 Kode Program

Berikut adalah kode program lengkap pada sel Jupyter Notebook Tes_IMCLab_naik_turun.ipynb.

```
# ============================================================
# Program Kontrol Motor ESP32 via Serial (Auto-Detect Port)
# Fitur: ON/OFF + Naik/Turun kecepatan (PWM) - Mode Interaktif
# ============================================================

import serial
import serial.tools.list_ports
import sys
import time

# ============================================
# KONFIGURASI
# ============================================
BAUDRATE = 115200

ESP32_KEYWORDS = [
    "cp210",
    "cp2102",
    "cp210x",
    "ch340",
    "ch341",
    "ch34x",
    "ftdi",
    "ft232",
    "usb-serial",
    "usb serial",
    "usb uart",
    "uart bridge",
    "silicon labs",
    "espressif",
    "esp32",
]

ESP32_VID_PID = {
    (0x10C4, 0xEA60),
    (0x10C4, 0xEA70),
    (0x1A86, 0x7523),
    (0x1A86, 0x55D4),
    (0x0403, 0x6001),
    (0x0403, 0x6015),
    (0x303A, 0x1001),
}

DUTY_STEP = 10  # selisih kenaikan/penurunan di sisi Python
(informatif)


# ============================================
# Auto-detect port ESP32
# ============================================
def list_serial_ports():
  ports = serial.tools.list_ports.comports()
  if not ports:
    print("Tidak ada port serial yang terdeteksi.")
    return []
  print("Port serial yang tersedia:")
  for p in ports:
    vid_pid = ""
    if p.vid is not None and p.pid is not None:
      vid_pid = f" [VID:PID={p.vid:04X}:{p.pid:04X}]"
    print(f"  - {p.device} : {p.description}{vid_pid}")
  return list(ports)
```

```
def deteksi_port_esp32(verbose=True):
  ports = serial.tools.list_ports.comports()
  if not ports:
    if verbose:
      print("[!] Tidak ada port serial yang terdeteksi.")
    return None

  kandidat = []
  for p in ports:
    desc = (p.description or "").lower()
    mfg = (p.manufacturer or "").lower()
    hwid = (p.hwid or "").lower()
    teks = f"{desc} {mfg} {hwid}"

    skor = 0
    if p.vid is not None and p.pid is not None:
      if (p.vid, p.pid) in ESP32_VID_PID:
        skor += 100
    for kw in ESP32_KEYWORDS:
      if kw in teks:
        skor += 50
        break
    if "bluetooth" in teks:
      skor -= 200

    if skor > 0:
      kandidat.append((skor, p))

  if not kandidat:
    non_bt = [
        p for p in ports if "bluetooth" not in (p.description or
"").lower()
    ]
    if len(non_bt) == 1:
      if verbose:
        print(f"[i] Hanya ada 1 port non-Bluetooth:
{non_bt[0].device}")
      return non_bt[0].device
    if verbose:
      print("[!] Port ESP32 tidak dapat dideteksi otomatis.")
    return None

  kandidat.sort(key=lambda x: x[0], reverse=True)
  if verbose:
    print("Kandidat port ESP32 (skor tertinggi = paling mungkin):")
    for skor, p in kandidat:
      print(f"  [{skor:>4}] {p.device} : {p.description}")
  return kandidat[0][1].device


# ============================================
# Koneksi serial
# ============================================
def buka_koneksi(port, baudrate=115200):
  try:
    ser = serial.Serial(port, baudrate, timeout=1)
    time.sleep(2)
    print(f"Terhubung ke ESP32 di {port} @ {baudrate} baud")
    return ser
  except serial.SerialException as e:
    print(f"Gagal membuka port {port}: {e}")
    return None


# ============================================
# Kirim perintah & baca balasan
# ============================================
def kirim_perintah(ser, perintah: str):
  """Mengirim perintah ke ESP32 (diakhiri newline).

  Perintah valid: '0', '1', '+', '-', 'PWM:xxx'
  """
  ser.write((perintah + "\n").encode("utf-8"))
  ser.flush()
  time.sleep(0.15)
```

```
  # Baca semua baris balasan yang tersedia
  while ser.in_waiting > 0:
    balasan = ser.readline().decode("utf-8", errors="ignore").strip()
    if balasan:
      print(f"ESP32: {balasan}")
    time.sleep(0.05)


# Wrapper perintah
def motor_on(ser):
  print(">> Motor ON")
  kirim_perintah(ser, "1")


def motor_off(ser):
  print(">> Motor OFF")
  kirim_perintah(ser, "0")


def kecepatan_naik(ser, step=DUTY_STEP):
  """Naikkan kecepatan motor sebanyak `step` (default 10)."""
  print(f">> Kecepatan +{step}")
  kirim_perintah(ser, "+")


def kecepatan_turun(ser, step=DUTY_STEP):
  """Turunkan kecepatan motor sebanyak `step` (default 10)."""
  print(f">> Kecepatan -{step}")
  kirim_perintah(ser, "-")


def set_pwm(ser, duty: int):
  """Set duty cycle langsung (0-255)."""
  duty = max(0, min(255, int(duty)))
  print(f">> Set PWM = {duty}")
  kirim_perintah(ser, f"PWM:{duty}")


# ============================================
# PROGRAM UTAMA
# ============================================
if __name__ == "__main__":
  print("=" * 50)
  list_serial_ports()
  print("=" * 50)

  PORT = deteksi_port_esp32()
  if PORT is None:
    print("Program dihentikan: port ESP32 tidak ditemukan.")
    sys.exit(1)
  print(f"\n>> Port ESP32 terpilih: {PORT}")

  esp32 = buka_koneksi(PORT, BAUDRATE)

  if esp32 is None:
    print("Koneksi gagal. Periksa kembali PORT dan kabel USB.")
  else:
    try:
      # ---- Mode Interaktif Langsung Aktif ----
      print("\n" + "=" * 50)
      print("MODE KONTROL MOTOR INTERAKTIF SIAP")
      print("=" * 50)
      print("  '1'     = Motor ON")
      print("  '0'     = Motor OFF")
      print("  '+'     = Naikkan kecepatan (+10)")
      print("  '-'     = Turunkan kecepatan (-10)")
      print("  'p<val>'= Set PWM langsung (contoh: p150)")
      print("  'q'     = Keluar / Exit")
      print("=" * 50)

      while True:
        cmd = input("\nMasukkan Perintah: ").strip()

        if cmd.lower() == "q":
          print("Keluar dari program...")
          break
```

```
        elif cmd == "1":
          motor_on(esp32)
        elif cmd == "0":
          motor_off(esp32)
        elif cmd == "+":
          kecepatan_naik(esp32)
        elif cmd == "-":
          kecepatan_turun(esp32)
        elif cmd.lower().startswith("p") and cmd[1:].isdigit():
          set_pwm(esp32, int(cmd[1:]))
        else:
          print(
              "[!] Perintah tidak valid! Gunakan: 1, 0, +, -, p<angka>
(misal:"
              " p150), atau q."
          )

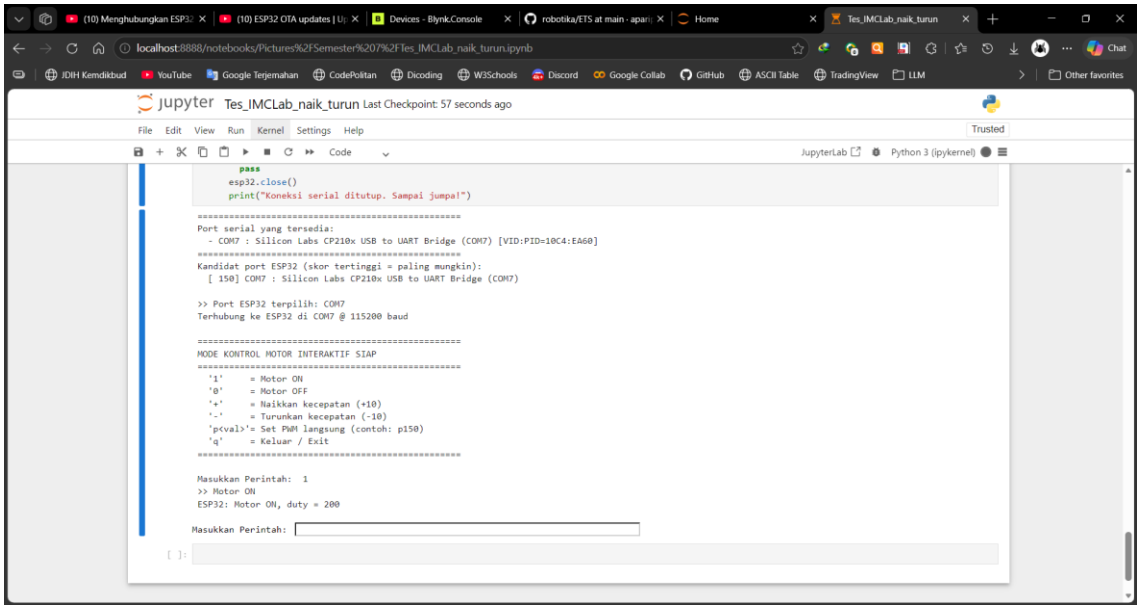
    except KeyboardInterrupt:
      print("\n[!] Dihentikan oleh pengguna (Ctrl+C).")

    finally:
      try:
        motor_off(esp32)
      except Exception:
        pass
      esp32.close()
      print("Koneksi serial ditutup. Sampai jumpa!")
```

BAB II HASIL DAN PEMBAHASAN

2.1 Deteksi Port Otomatis dan Inisialisasi Program

Sel program dijalankan melalui Jupyter Notebook yang diakses pada alamat localhost:8888 dengan kernel Python 3 (ipykernel).

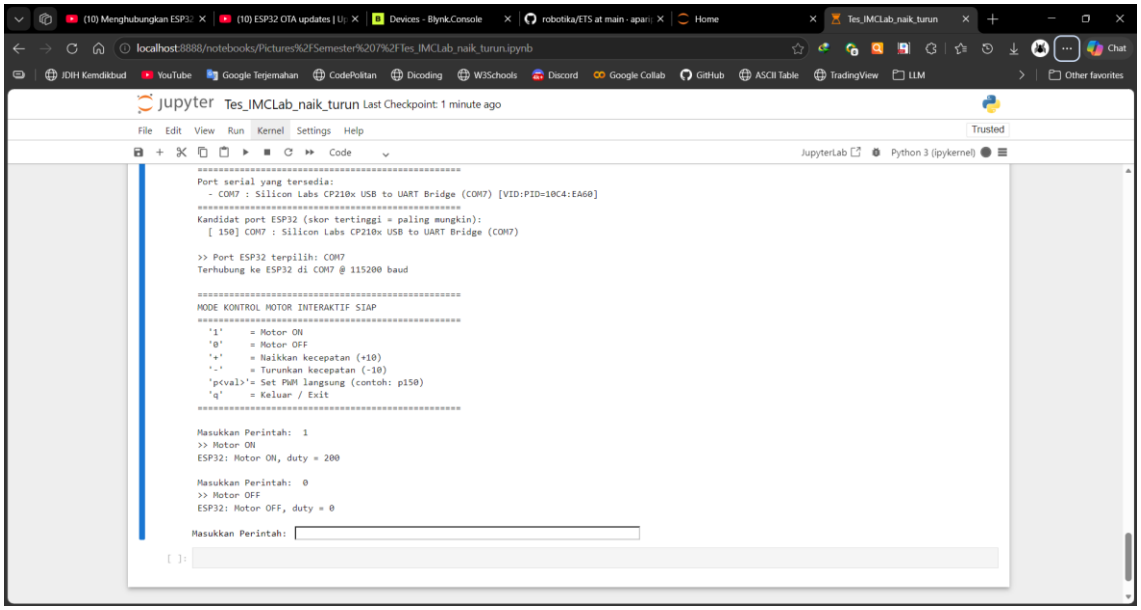


Gambar 1. Sel program dan proses deteksi port otomatis pada Jupyter Notebook

Keluaran program menampilkan satu port yang tersedia, yaitu COM7 dengan deskripsi Silicon Labs CP210x USB to UART Bridge dan identitas VID:PID = 10C4:EA60. Pasangan tersebut terdaftar pada himpunan ESP32_VID_PID sehingga memperoleh skor 100, ditambah 50 dari pencocokan kata kunci deskripsi, menghasilkan skor akhir 150. Karena merupakan kandidat dengan skor tertinggi, COM7 dipilih secara otomatis dan koneksi berhasil dibuka pada 115200 baud, dilanjutkan dengan tampilnya menu mode kontrol motor interaktif.

2.2 Pengujian Perintah ‘1’ dan ‘0’

Pengujian pertama dilakukan dengan memasukkan perintah ‘1’ lalu ‘0’ pada kotak input interaktif.

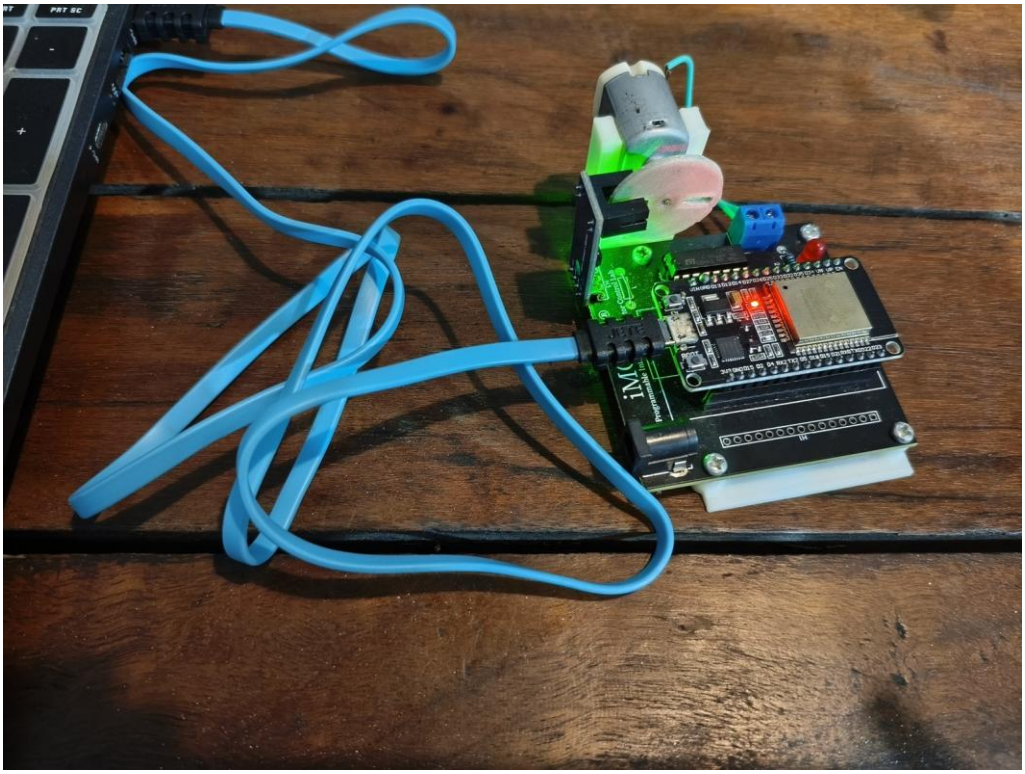


Gambar 2. Hasil perintah ‘1’ dan ‘0’ pada Jupyter Notebook

Setiap perintah menghasilkan dua baris keluaran. Baris “>> Motor ON” berasal dari fungsi motor_on() pada sisi Python, sedangkan baris “ESP32: Motor ON, duty = 200” merupakan balasan dari firmware yang dibaca kembali oleh fungsi kirim_perintah(). Hal yang sama terjadi pada perintah ‘0’ yang menghasilkan “>> Motor OFF” dan balasan “ESP32: Motor OFF, duty = 0”. Adanya dua baris ini membuktikan komunikasi berlangsung dua arah, bukan hanya pengiriman satu arah.

2.3 Kondisi Perangkat Sebelum Roda Berputar (0)

Gambar berikut memperlihatkan kondisi fisik KIT IMCLAB setelah perintah ‘0’ dijalankan.

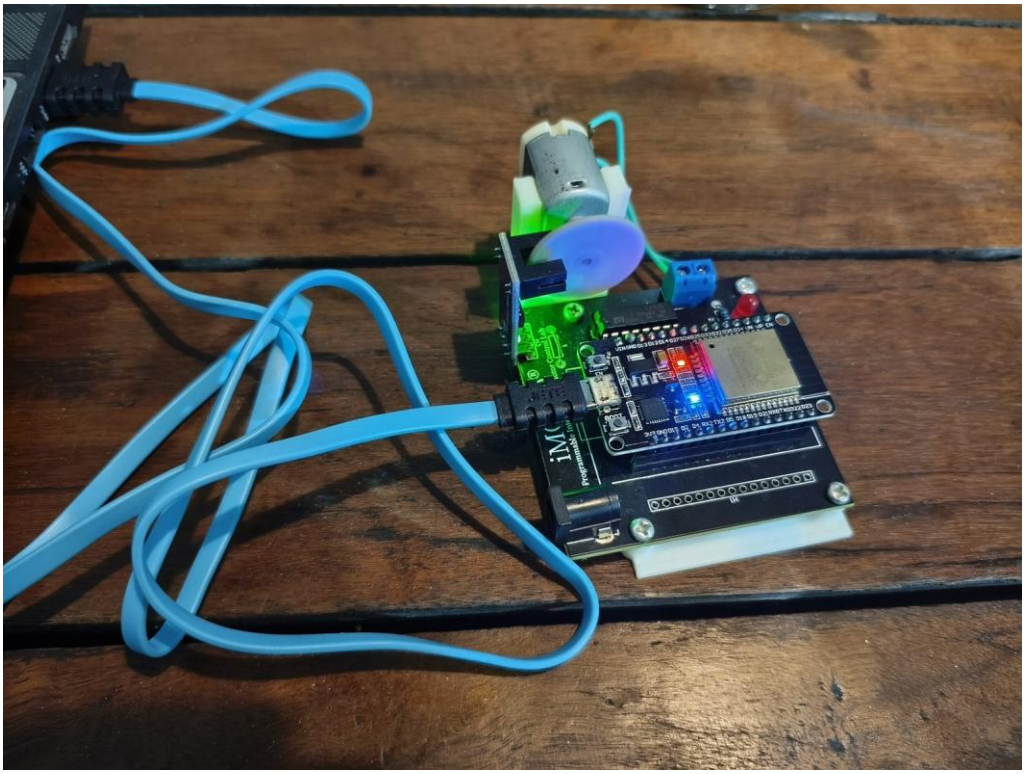


Gambar 3. Kondisi perangkat sebelum roda berputar (motor OFF)

Piringan pada poros motor terlihat jelas bentuknya dan tidak bergerak sehingga tertangkap tajam oleh kamera. LED indikator berwarna biru pada GPIO 2 dalam keadaan padam, sedangkan LED merah tetap menyala sebagai indikator catu daya. Kondisi ini sesuai dengan duty cycle bernilai 0 yang dilaporkan pada keluaran notebook.

2.4 Kondisi Perangkat Saat Roda Berputar (1)

Setelah perintah ‘1’ dikirimkan dari Jupyter Notebook, kondisi perangkat berubah seperti pada gambar berikut.

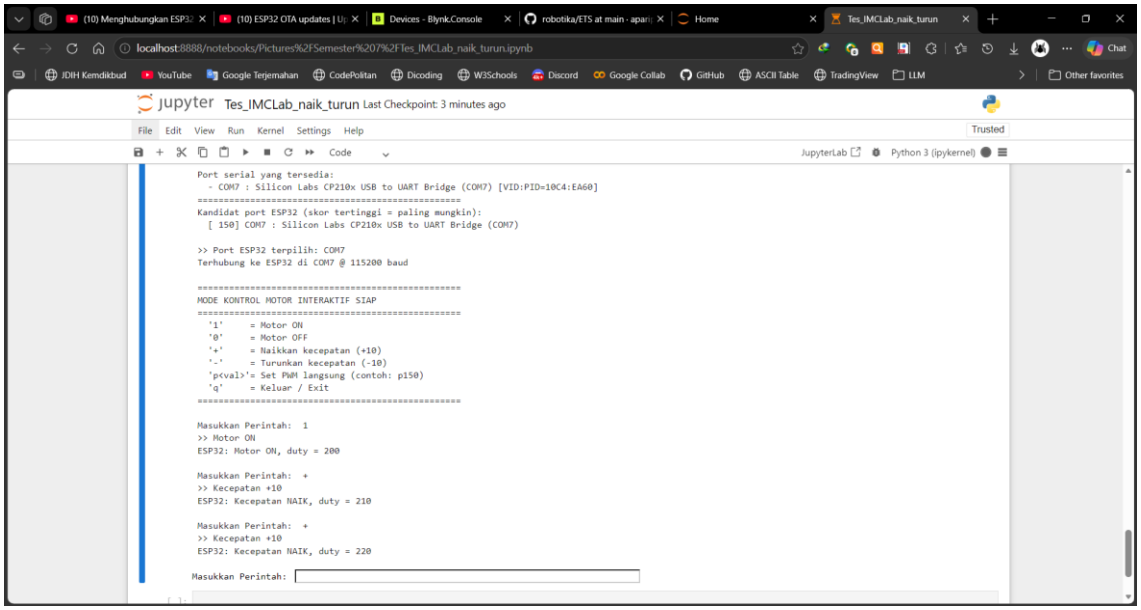


Gambar 4. Kondisi perangkat saat roda berputar (motor ON)

Piringan tampak kabur membentuk lingkaran penuh akibat efek gerak (motion blur) pada kamera, yang menandakan roda berputar dengan kecepatan tinggi. LED biru menyala bersamaan dengan berputarnya motor. Dengan demikian perintah yang dikirim dari Python benar-benar sampai ke perangkat keras dan bukan sekadar keluaran teks pada layar.

2.5 Pengujian Perintah ‘+’ (Menaikkan Kecepatan)

Setelah motor dinyalakan, perintah ‘+’ dimasukkan sebanyak dua kali berturut-turut.

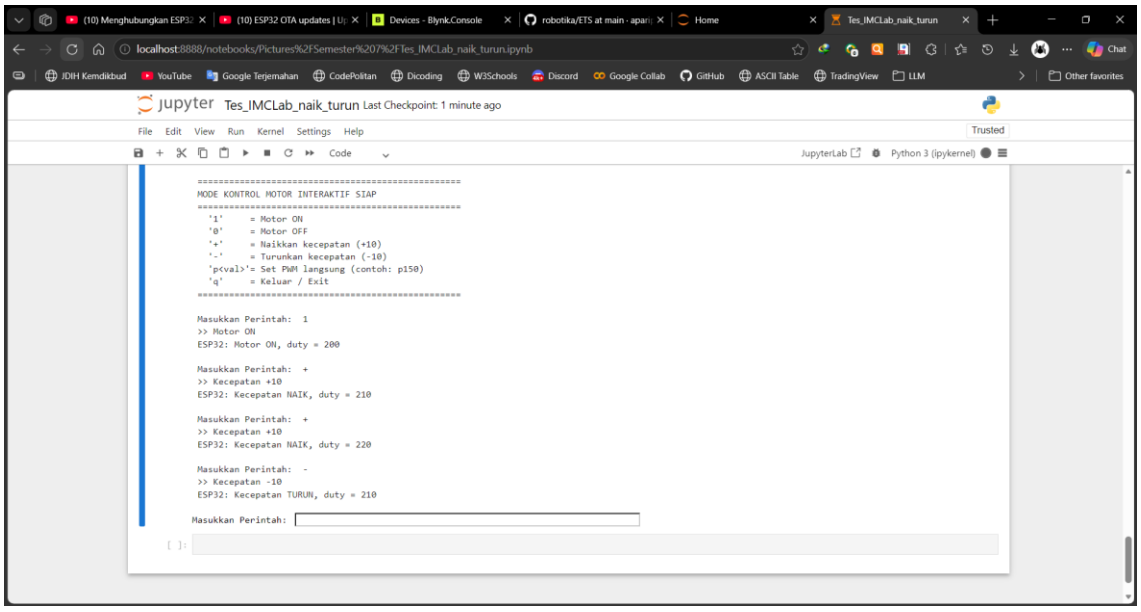


Gambar 5. Hasil perintah '+' sebanyak dua kali

Keluaran menampilkan ">> Kecepatan +10" diikuti balasan "ESP32: Kecepatan NAIK, duty = 210", kemudian pada perintah kedua menjadi duty = 220. Nilai DUTY_STEP pada sisi Python hanya bersifat informatif untuk pesan yang dicetak, karena penambahan nilai duty cycle yang sesungguhnya dilakukan oleh firmware ESP32 setelah menerima karakter '+'.

2.6 Pengujian Perintah '-' (Menurunkan Kecepatan)

Pengujian dilanjutkan dengan memasukkan perintah '-' setelah duty cycle berada pada nilai 220.

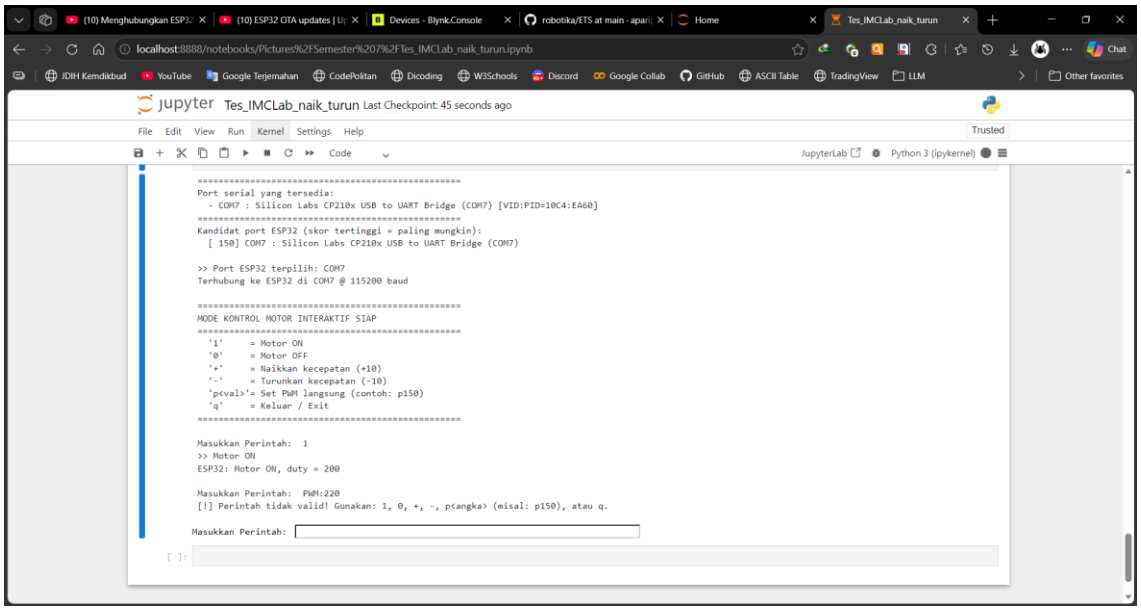


Gambar 6. Hasil perintah ‘-’ setelah kecepatan dinaikkan

Program menampilkan “>> Kecepatan -10” dan balasan “ESP32: Kecepatan TURUN, duty = 210”. Nilai duty cycle berkurang tepat 10 satuan sehingga langkah penurunan simetris dengan penambahan. Urutan lengkap yang terekam pada satu sesi adalah 200 → 210 → 220 → 210, yang menunjukkan status duty cycle tersimpan dengan benar di sisi firmware selama sesi berlangsung.

2.7 Pengujian Perintah ‘PWM:220’ (Sintaks Keliru)

Pengujian berikutnya dilakukan dengan memasukkan perintah PWM:220, yaitu sintaks yang berlaku pada Serial Monitor Arduino IDE.

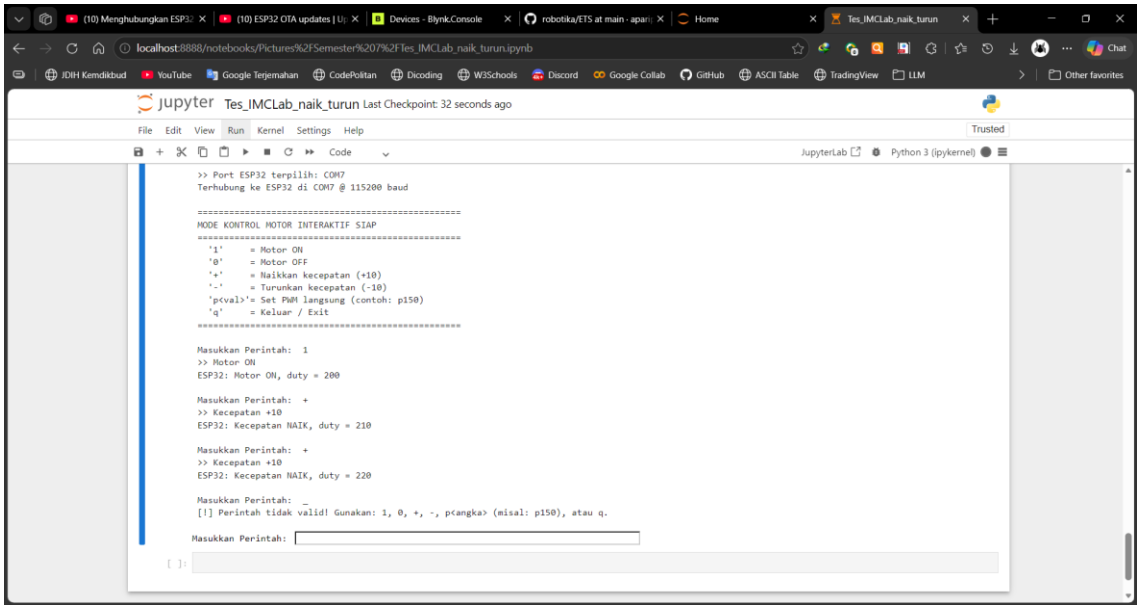


Gambar 7. Penolakan perintah ‘PWM:220’ oleh program Python

Program tidak meneruskan perintah tersebut dan menampilkan pesan “[!] Perintah tidak valid! Gunakan: 1, 0, +, -, p<angka> (misal: p150), atau q.”. Hal ini terjadi karena blok percabangan pada program Python hanya menerima masukan yang diawali huruf p dan diikuti angka, sesuai kondisi cmd.lower().startswith("p") and cmd[1:].isdigit(). Untuk menetapkan duty cycle 220, masukan yang benar adalah p220, yang kemudian diubah oleh fungsi set_pwm() menjadi perintah PWM:220 sebelum dikirim ke ESP32. Pengujian ini sekaligus menunjukkan bahwa program Python berfungsi sebagai lapisan validasi sebelum data diteruskan ke perangkat keras.

2.8 Pengujian Perintah Kosong (Validasi Masukan)

Pengujian terakhir dilakukan dengan menekan Enter tanpa mengisi perintah apa pun.



Gambar 8. Penolakan masukan kosong oleh validasi program

Program kembali menampilkan pesan peringatan yang sama dan tetap berada pada mode interaktif tanpa mengalami error ataupun berhenti. Dengan demikian perulangan while True beserta percabangan validasinya telah menangani masukan yang tidak dikenal dengan baik, sehingga sesi pengendalian motor tidak terganggu.

2.9 Rangkuman Hasil Pengujian

No	Masukan	Hasil Pengamatan
1	‘1’	“>> Motor ON” dan “ESP32: Motor ON, duty = 200”, roda berputar, LED biru menyala
2	‘+’	“ESP32: Kecepatan NAIK”, duty bertambah 200 → 210 → 220
3	‘-’	“ESP32: Kecepatan TURUN”, duty berkurang 220 → 210
4	‘0’	“ESP32: Motor OFF, duty = 0”, roda berhenti, LED biru padam
5	‘PWM:220’	Ditolak program Python, sintaks yang benar adalah p220
6	(kosong)	Ditolak program Python, mode interaktif tetap

No	Masukan	Hasil Pengamatan
		berjalan

Seluruh perintah yang valid memberikan keluaran yang konsisten antara pesan pada Jupyter Notebook dengan kondisi fisik perangkat, sedangkan masukan yang tidak valid berhasil ditangani tanpa menghentikan program.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan hasil pengujian, dapat disimpulkan beberapa hal sebagai berikut:

- Program Python pada Jupyter Notebook berhasil mendeteksi port ESP32 secara otomatis melalui penilaian VID:PID dan kata kunci deskripsi, dengan COM7 terpilih pada skor tertinggi 150.
- Koneksi serial pada 115200 baud menggunakan pustaka pyserial berhasil dibuka, sehingga perintah dapat dikirim dan balasan dari ESP32 dapat dibaca kembali.
- Perintah ‘1’, ‘0’, ‘+’, dan ‘-’ berfungsi sesuai rancangan, dengan perubahan duty cycle 200 → 210 → 220 → 210 dan 0 saat motor dimatikan.
- Perubahan duty cycle berpengaruh langsung pada perangkat keras, ditunjukkan oleh roda yang berputar dan LED biru menyala saat motor aktif, serta berhenti dan padam saat motor dimatikan.
- Masukan yang tidak sesuai, seperti PWM:220 maupun masukan kosong, berhasil ditolak oleh validasi program tanpa menghentikan mode interaktif; sintaks yang benar pada sisi Python adalah p<angka>.